Comparison Between Performance of Various Database Systems for Implementing a Language Corpus

Dimuthu Upeksha, Chamila Wijayarathna(✉), Maduranga Siriwardena, Lahiru Lasandun, Chinthana Wimalasuriya, N.H.N.D. de Silva, and Gihan Dias

Department of Computer Science and Engineering,
University of Moratuwa, Sri Lanka
{upeksha.10,cdwijayarathna.10,maduranga.10,lasandun.10,
chinthana,nisansadds,gihan}@cse.mrt.ac.lk
http://www.cse.mrt.ac.lk

Abstract. Data storage and information retrieval are some of the most important aspects when it comes to the development of a language corpus. Currently most corpora use either relational databases or indexed file systems. When selecting a data storage system, most important facts to consider are the speeds of data insertion and information retrieval. Other than the aforementioned two approaches, currently there are various database systems which have different strengths that can be more useful. This paper compares the performance of data storage and retrieval mechanisms which use relational databases, graph databases, column store databases and indexed file systems for various steps such as inserting data into corpus and retrieving information from it, and tries to suggest an optimal storage architecture for a language corpus.

Keywords: Corpus · Relational Databases · NoSQL · Graph Database · Neo4j · Index file system · Apache Solr · Column stores · Cassandra

1 Introduction

Today, corpus based approach can be identified as the state of the art methodology in language studying for both prominent and inconspicuous languages in the world. Corpus based approach mines new knowledge on a language by answering two main questions:

1. What particular patterns are associated with lexical or grammatical features of the language?
2. How do these patterns differ within varieties and registers?

According to Bennet, et al. [2]: A corpus is a principled collection of authentic texts stored electronically that can be used to discover information about language that may not have been noticed through intuition alone.

From the top view architecture, a corpus may have three main components. And those are a mechanism to insert new data into the corpus, a data storage

© Springer International Publishing Switzerland 2015
S. Kozielski et al. (Eds.): BDAS 2015, CCIS 521, pp. 82–91, 2015.
DOI: 10.1007/978-3-319-18422-7_7

and information retrieval system, and data visualization tools. Among these components, data storage system is the most vital part, because the performance of data insertion and retrieval mainly depends on it. In this study we have carried out a comprehensive comparison on a set of widely used database systems and technologies in their role of being a data storage component for a corpus in order to find an optimal solution with optimal performance.

The remaining five sections of this paper describe our study. First in section 2, we briefly look at the storage mechanisms that have been used in existing corpora and the data storage technologies we are going to analyse with relevant architectures. Section 3 describes the method we are using to compare those storage mechanisms. Then in section 4 we present the results and observations of our study and finally in section 5, we present the drawn conclusions.

2 Suggested Solutions for Comparison

According to the literature there are two main storage systems used in the existing corpora. They are relational databases and indexed file systems.

Popular corpora like Corpus of Contemporary American English (COCA) and Corpus del Espaol use relational databases. Davies [3,4] has described the relational database model used in COCA and the details of database model used in Corpus del Espaol.

On the other hand British National Corpus (BNC), uses indexed file systems where data is stored as XML like files which follows a scheme known as the Corpus Data Interchange Format (CDIF).Aston, et al. [1] describes about the storage mechanism of BNC.

The idea of this study is to compare existing database systems and to come up with strengths and weaknesses of each system as a candidate for a storage mechanism of a language corpus.

In this study we are using H2 database as a relational database system because its licensing permits us to publish benchmarks and according to H2 benchmark tests [5], H2 has performed better than other relational databases such as MySQL, PostgreSQL, Derby and HSQLDB that allow publishing benchmarks. We did not do these performance testings on relational databases such as Oracle, MS SQL Server and DB2 because their licenses are not compatible with benchmark publishing.

We used Apache Solr which is powered by Apache Lucene as an indexed file system. We selected Solr because Lucene is a widely used text search engine and Solr is licensed under Apache License which allows us to do benchmark testing on that.

When storing details about words, bigrams, trigrams and sentences, one of the biggest challenges faced is storing the relationships between each of these entities. One way to represent relationships between entities is to use a graph. Hence, we also considered graph databases in our study as a candidate for the optimal storage system. Currently there are several implementations of graph databases such as Neo4j, OrientDB, Titan and DEX. In our study we used

Neo4j as our graph database system because, it has been identified by Jouili and Vansteenberghe [6] that Neo4j has performed better than the other graph databases.

Column databases improves the performance of information retrieval at the cost of higher insertion time and lack of consistency. Since one of our main goals is fast information retrieval, we considered column databases as a candidate too. We used Cassandra since it has proven to give higher throughput than other widely used column databases [7].

3 Method of Comparison

This section describes the method that we used for the comparison and how we set up each database system. All codes and dataset used for this analysis is available on a Github repository (goo.gl/JyoHRa).

3.1 Benchmarking Conditions

Dataset - The dataset we used in this study contains data which were crawled from online sources that are written in the Sinhala language. The final dataset consisted of 5 million word tokens, 400000 sentences and 20000 posts.

Testing Environment - All tests were run in a 2.4 GHz core i7 machine with 12GB of physical memory and a standard hard disk (spinning disk). Operating system was Ubuntu 14.04 with java version 1.8.0_05.

Data Insertion Mechanism - For every database system mentioned in the previous section, words were added in 50 iterations in which each iteration added 100,000 words with relevant sentences and posts.

Data Retrieval Mechanism - At the end of each iteration mentioned above, queries to retrieve information for following scenarios were passed to each database. Same query was executed 6 times in each iteration and selected the median of recorded values.

1. Get frequency of a given word in corpus using same word for every database system
2. Get list of frequencies of a given word in different time periods and different categories
3. Get most frequently used 10 words in a given time period or a given category
4. Get latest 10 posts that include a given word
5. Get latest 10 posts that include a given word in a given time period or a given category
6. Get 10 words which are most frequent as the last word of a sentence
7. Get the frequency of a given bigram in given time period and a given category
8. Get the frequency of a given trigram in given time period and a given category

9. Get most frequent bigrams
10. Get most frequent trigrams
11. Get most frequent word after a given word
12. Get most frequent word after a given bigram

3.2 Method of Evaluating H2 DB

We used H2 version 1.3.176 for this performance analysis. Evaluation was done using JDBC driver version 1.4.182. Cache was set to 0 Mb. We used relational schema shown in Fig. 1.

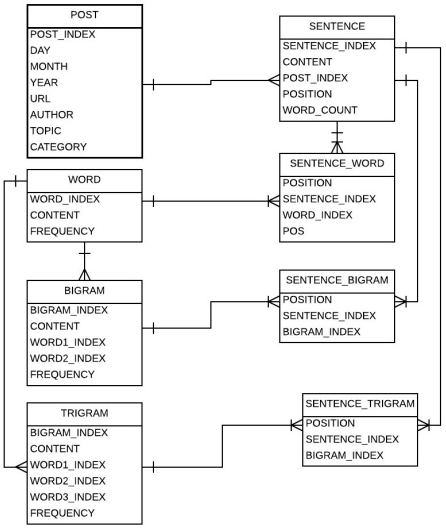


Fig. 1. Schema used in H2

We used below indexes for H2 database.

1. Index on CONTENT column of the WORD table
2. Index on FREQUENCY column of the WORD table
3. Index on YEAR column of the POST table
4. Index on CATEGORY column of the POST table
5. Index on CONTENT column of the BIGRAM table
6. Index on WORD1_INDEX column of the BIGRAM table
7. Index on WORD2_INDEX column of the BIGRAM table
8. Index on CONTENT column of the TRIGRAM table
9. Index on WORD1_INDEX column of the TRIGRAM table
10. Index on WORD2_INDEX column of the TRIGRAM table
11. Index on WORD3_INDEX column of the TRIGRAM table
12. Index on POSITION column of the SENTENCE_WORD table

When inserting the data to the database we dropped all the indexes except the one on CONTENT column of the WORD table. The indexes were recreated when retrieving information.

3.3 Method of Evaluating Neo4j

We used Neo4j community distribution 2.1.2 licensed under GPLv3. Evaluation was done using java embedded Neo4j database mode with heap size of 4048 MB which can pass database queries through a java API. We measured performance using both warm cache mode and cold cache mode. In warm cache mode, all caches were cleared and a set of warm up queries were run before running each actual query. In cold cache mode, queries were run with an empty cache. We did batch insertion through csv import function with pre-generated csv files.

We used graph structure in Fig. 2 to evaluate the performance.

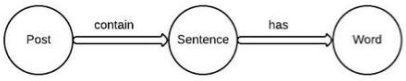


Fig. 2. Graph structure used in Neo4j

Properties of Nodes

1. Post - post_id, topic, category, year, month, day, author and url
2. Sentence - sentence_id, word_count
3. Word - word_id, content, frequency

Properties of Relationships

1. Contain - position
2. Has - position

Procedure of Data Insertion. We used csv file formatted database dumps of relational database in-order to insert data into Neo4j using its csv import function.

1. Word.csv includes word_id, content and frequency
2. Post.csv includes post_id, topic, category, year, month, day, author and url
3. Sentence.csv includes sentence_id, post_id, position and word_count
4. Word_Sentence.csv includes word_id, sentence-id and position

Evaluations were done for both data insertion times and data retrieval times. Data retrieval time was calculated by measuring the execution time for each cypher query. For each cypher query, two time measurements were recorded for warm cache mode and cold cache mode.

3.4 Method of Evaluating Apache Solr

We used Apache Solr version 4.10.2. Cache sizes of LRUCache, FastLRUCache and LFUCache were set to 0 in solrconfig.xml and autoWarmCount for each cache was set to 0.

The following field types were defined to store data in Solr.

1. text_general - This was implemented using solr TextField data typeand the standard tokenizer was used.
2. text_shingle_2 - This was implemented using solr TextField data type and ShingleFilterFactory with minShingleSize="2" and maxShingleSize="2".
3. text_shingle_3 - This was implemented using solr TextField data type and ShingleFilterFactory with minShingleSize="2" and maxShingleSize="2".

To evaluate the performance, fields shown in table 1 were added to documents.

Table 1. Data types definition of Solr search engine

Fieldname	*Fieldtype*	*Indexed*	*Sorted*
id	string	true	true
date	date	true	true
topic	text_general	true	true
author	text_general	true	true
link	text_general	false	true
context	text_general	true	true
content_shingled_2	text_shingle_2	true	true
content_shingled_3	text_shingle_3	true	true

For each doc, an unique id of 7 characters was generated. All fields except link were indexed and all the fields were sorted.

3.5 Method of Evaluating Cassandra

We used Apache Cassandra version 2.1.1. Both Key Caching and Row Caching were disabled when retrieving information from the database.

Cassandra uses a query based data modeling where it maintains different column families to address querying needs. Hence, when designing our database we also maintained different column families for different querying needs and consistency among them was maintained at the application level that we used to insert data.

Following are the column families we used, with the indexes used in each one of them.

1. word_frequency (id bigint, content varchar, frequency int, PRIMARY KEY (content))

2. word_time_frequency (id bigint, content varchar, year int, frequency int, PRIMARY KEY (year, content))
3. word_time_inv_frequency (id bigint, content varchar, year int, frequency int, PRIMARY KEY((year) , frequency, content))
4. word_usage (id bigint, content varchar, sentence varchar, date timestamp, PRIMARY KEY (content, date, id))
5. word_yearly_usage (id bigint, content varchar, sentence varchar, position int, postname text, year int, day int, month int, date timestamp, url varchar, author varchar, topic varchar, category int, PRIMARY KEY ((content, year, category) ,date,id))
6. word_pos_frequency (id bigint, content varchar, position int, frequency int, PRIMARY KEY ((position), frequency, content))
7. word_pos_id (id bigint, content varchar, position int, frequency int, PRIMARY KEY (position, content))
8. bigram_with_word_frequency (id bigint, word1 varchar, word2 varchar, frequency int, PRIMARY KEY (word1, frequency, word2))
9. bigram_with_word_id (id bigint, word1 varchar, word2 varchar, frequency int, PRIMARY KEY (word1, word2))
10. bigram_time_frequency (id bigint, bigram varchar, year int, frequency int, PRIMARY KEY (year, bigram))
11. trigram_time_frequency (id bigint, trigram varchar, year int, frequency int, PRIMARY KEY (year, trigram))
12. bigram_frequency (id bigint, content varchar, frequency int, category int, PRIMARY KEY (category, frequency, content))
13. bigram_id (id bigint, content varchar, frequency int, PRIMARY KEY (content))
14. trigram_frequency (id bigint, content varchar, frequency int, category int, PRIMARY KEY (category, frequency, content))
15. trigram_id (id bigint, content varchar, frequency int, PRIMARY KEY (content))
16. trigram_with_word_frequency (id bigint, word1 varchar, word2 varchar, word3 varchar, frequency int, PRIMARY KEY ((word1, word2), frequency, word3))
17. trigram_with_word_id (id bigint, word1 varchar, word2 varchar, word3 varchar, frequency int, PRIMARY KEY (word1, word2, word3))

4 Observations and Results

Fig. 3 shows the comparison between the data insertion time for each data storage mechanism. Fig. 4 and Fig. 5 illustrate the comparison between times taken for the each information retrieval scenario for each data storage mechanism. When plotting graphs, we have ignored relatively higher values to increase the clarity of graphs.

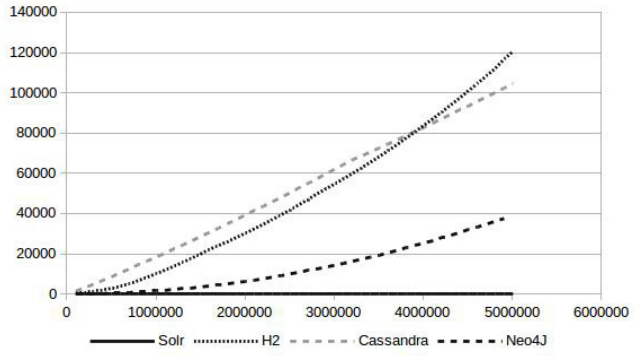


Fig. 3. Data insertion time in each data storage system

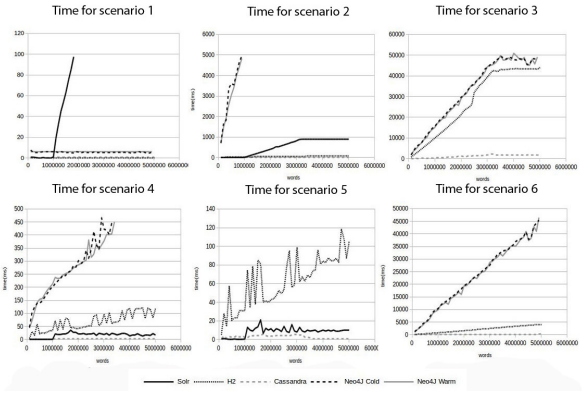


Fig. 4. Data retrieval time for each scenario in each data storage system - part 1

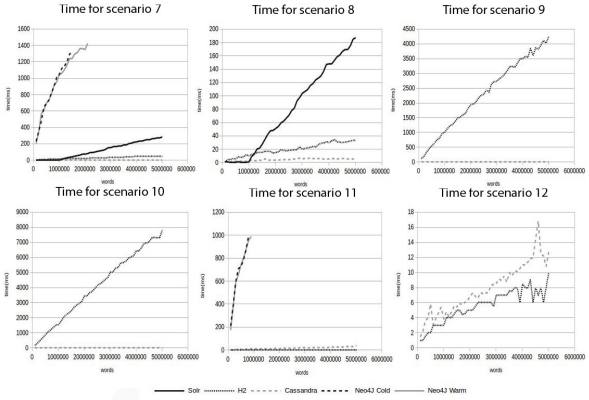


Fig. 5. Data retrieval time for each scenario in each data storage system - part 2

Normalized Time Measurements for Data Insertion. All data insertion time measurements of each database were normalized with respect to data insertion time measurements of H2 database.

Normalized time for data insertion in database

$$DB_i = \frac{1}{number_of_iterations} \sum \frac{time_to_insert_data_to_DB_i}{time_to_insert_data_t_H2} \tag{1}$$

Table 2. Normalized Time for Data Insertion in Each Data Storage System

H2	*Solr*	*Neo4j*	*Cassandra*
1	0.00047	0.2333	1.5306

Normalization of Time Measurements for Data Retrieval for Each Scenario. All time measurements were normalized with respect to time measurements of H2 database.

Normalized time for query q in database

$$DB_i = \frac{1}{number_of_iterations} \sum \frac{measured_time_of_DB_i_iteration_k_for_query_q}{measured_time_of_H2_iteration_k_for_query_q} \tag{2}$$

Table 3. Normalized data retrieval time for each scenario in each data storage system

Normalized time	*Q*1	*Q*2	*Q*3	*Q*4	*Q*5	*Q*6	*Q*7	*Q*8	*Q*9	*Q*10	*Q*11	*Q*12
H2	1	1	1	1	1	1	1	1	1	1	1	1
Solr	192	8.34	N/S	0.26	0.13	N/S	3.25	3.23	N/S	N/S	N/S	N/S
Neo4j Cold Cache	5.3	224	1.35	5.84	197	12.4	60	586	T/E	T/E	1228	2464
Neo4j Warm Cache	6	221	1.32	5.52	192	12	56	566	T/E	T/E	1141	2286
Cassandra	0.7	0.02	0.04	0.04	0.06	0.03	0.1	0.23	0.002	0.001	11.0	1.25

Reasons for having N/S and T/E values for the entries in table 3 are:

1. N/S (Not Support) - Storage system or the schema does not support data retrieval of such scenarios.
2. T/E (Time Exceeds) - Data retrieval time takes more than 120 seconds in such scenarios

5 Conclusion

Based on our observations, in data insertion point of view, Solr performed better than the other three databases with a significant amount of time gap (Solr : H2 : Neo4j : Cassandra = 1 : 2127 : 495 : 3256). That means Solr is approximately

2127 times faster than H2, 495 times faster than Neo4j and 3256 times faster than Cassandra with respect to data insertion time.

From data retrieval point of view Cassandra performed better than other databases. In couple of scenarios H2 outperformed Cassandra. But in those scenarios, Cassandra also showed a considerably good speed. Neo4j did not perform well in any scenario and hence it is not suitable to create a structured dataset such as a language corpus. Solr also marked a decent performance in some scenarios but there were issues in implementing some scenarios because its underlying indexing mechanism did not provide support to do those.

Only issue with Cassandra was its less flexibility in supporting new queries. If we had another information need other than the above mentioned scenarios, we would have to create new column families in Cassandra that support given information need and insert data from the beginning which is a very expensive process.

In conclusion Cassandra can be named as a good candidate for a data storage system of a language corpus because

1. Data insertion time of Cassandra is linear which can be very effective in a long term data insertion process
2. Performed well in 10 out of all 12 scenarios for information retrieval.

References

1. Aston, G., Burnard, L.: The BNC Handbook:Exploring the British National Corpus with SARA, http://corpus.leeds.ac.uk/teaching/aston-burnard-bnc.pdf
2. Bennet, G.R.: Using Corpora in the Language Learning Classroom, http://www.international.ucla.edu/media/files/Using-corpora-in-the-language-learning-classroom-Corpus-linguistics-for-teachers-my-atc.pdf
3. Davies, M.: The advantage of using relational databases for large corpora. International Journal of Corpus Linguistics 10(3), 307–335 (2005)
4. Davies, M.: The 385+ million word corpus of contemporary American English (1990–2008+) design, architecture, and linguistic insights. International Journal of Corpus Linguistics 14(2), 159–191 (2009)
5. H2: Performance, http://www.h2database.com/html/performance.html
6. Jouili, S., Vansteenberghe, V.: The advantage of using relational databases for large corpora. In: International Conference on Social Computing (SocialCom), pp. 708–715 (2013)
7. Rabl, T.M., et al.: Solving big data challenges for enterprise application performance management. PVLDB, 1724–1735 (2010)